all layers in a block through batching, the total per-layer memory access cost remains only $(\frac{N}{S}+3)d$ reads and $2d$ writes (Table 1). This is substantially lower than the residual-stream I/O of prior residual generalizations such as (m)HC under typical settings. In practice, Phase 1 can also partially overlap with the computation of the first layer in the block, further reducing its wall-clock impact. As a result, the end-to-end inference latency overhead is less than 2% on typical inference workloads.

Table 1: Memory access cost per token per layer incurred by the residual mechanism under each scheme. The internal I/O of the layer function f_l is excluded. For AttnRes, both Full and Block variants use the two-phase inference schedule described in Appendix B; amortized costs are averaged over N layers within a block. Typical values: L=128, N=8, S=L/N=16, m=4.

		Operation	Read	Write	Total I/O	
					Symbolic	Typical
Standard Residuals		Residual Merge	$2d$	d	$3d$	$3d$
mHC (m streams)		Compute $\boldsymbol{\alpha}_l$, $\boldsymbol{\beta}_l$, $\mathbf{A}_l$	md	$m^2{+}2m$	$(8m{+}2)d{+}2m^2{+}4m$	$34d$
		Apply $\boldsymbol{\alpha}_l$	$md{+}m$	d		
		Apply $\boldsymbol{\beta}_l$	$d{+}m$	md		
		Apply $\mathbf{A}_l$	$md{+}m^2$	md		
		Residual Merge	$2md$	md		
AttnRes	Full	Phase 1 (amortized)	$(N{-}1)d$	d	$(S{+}N)d$	$24d$
		Phase 2	$(S{-}1)d$	d		
	Block	Phase 1 (amortized)	$\frac{N}{S}d$	d	$\left(\frac{N}{S}{+}5\right)d$	$5.5d$
		Phase 2	$3d$	d		

Memory-efficient prefilling. Storing block representations during prefilling requires $N \cdot T \cdot d$ elements, which incurs 15 GB of memory for a 128K-token sequence with 8 blocks. We mitigate this by sharding these representations along the sequence dimension across P tensor-parallel devices, allowing Phase 1 to execute independently on local sequence shards. The Phase 2 online-$\mathrm{softmax}$ merge then integrates into the standard TP all-reduce communication path: the output is reduce-scattered, merged locally, and reconstructed via all-gather, naturally admitting kernel fusion with operations like $\mathrm{RMSNorm}$. This reduces the per-device memory footprint to $N \cdot (T/P) \cdot d$—lowering the 128K-context example from 15 GB to roughly 1.9 GB per device. Combined with chunked prefill (e.g., 16K chunk size), the overhead further reduces to under 0.3 GB per device.

5 Experiments

Architecture Details. Our architecture is identical to Kimi Linear [69], a Mixture-of-Experts (MoE) Transformer following the Moonlight [28] / DeepSeek-V3 [9] design, which interleaves Kimi Delta Attention (KDA) and Multi-Head Latent Attention (MLA) layers in a 3:1 ratio, each followed by an MoE feed-forward layer. The only modification is the addition of AttnRes to the residual connections; all other components (model depth, hidden dimensions, expert routing, and MLP structure) remain unchanged. AttnRes introduces only one $\mathrm{RMSNorm}$ and one pseudo-query vector $\boldsymbol{w}_l \in \mathbb{R}^d$ per layer, amounting to a negligible fraction of the total parameter count. Crucially, all pseudo-query vectors must be initialized to zero. This ensures that the initial attention weights $\alpha_{i\to l}$ are uniform across source layers, which reduces AttnRes to an equal-weight average at the start of training and prevents training volatility, as we validated empirically.

5.1 Scaling Laws

We sweep five model sizes (Table 2) and train three variants per size: a PreNorm baseline, Full AttnRes, and Block AttnRes with ≈ 8 blocks. They are trained with an 8192-token context window and a cosine learning rate schedule. Within each scaling law size group, all variants share identical hyperparameters selected under the baseline to ensure fair comparison; this setup intentionally favors the baseline and thus makes the comparison conservative. Following standard practice, we fit power-law curves of the form $\mathcal{L} = A \times C^{-\alpha}$ [22, 15], where $\mathcal{L}$ is validation loss and C is compute measured in PFLOP/s-days.

Scaling Behavior. Fig. 4 presents the fitted scaling curves. The Baseline follows $\mathcal{L} = 1.891 \times C^{-0.057}$, while Block AttnRes fits $\mathcal{L} = 1.870 \times C^{-0.058}$, and Full AttnRes fits $\mathcal{L} = 1.865 \times C^{-0.057}$. All three variants exhibit a similar slope, but AttnRes consistently achieves lower loss across the entire compute range. Based on the fitted curves, at 5.6

Table 2: Baseline vs Block AttnRes ($N = 8$) vs Full AttnRes vs mHC(-lite) [64]: Model configurations, Hyperparameters, and Validation Loss.

# Act. Params†	Tokens	L_b	H	d_{model}	d_{ff}	lr	batch size‡	Val. Loss			
								Baseline	Block AttnRes	Full AttnRes	mHC(-lite)
194M	38.7B	12	12	896	400	2.99×10^{-3}	192	1.931	1.909	**1.899**	1.906
241M	45.4B	13	13	960	432	2.80×10^{-3}	256	1.895	1.875	1.874	**1.869**
296M	62.1B	14	14	1024	464	2.50×10^{-3}	320	1.829	1.809	**1.804**	1.807
436M	87.9B	16	16	1168	528	2.20×10^{-3}	384	1.766	1.746	**1.737**	1.747
528M	119.0B	17	17	1264	560	2.02×10^{-3}	432	1.719	1.693	**1.692**	1.694

† Denotes the number of activated parameters in our MoE models, excluding embeddings.
‡ All models were trained with a context length of 8192.
⋆ $L_b = L/2$ denotes the number of Transformer blocks.

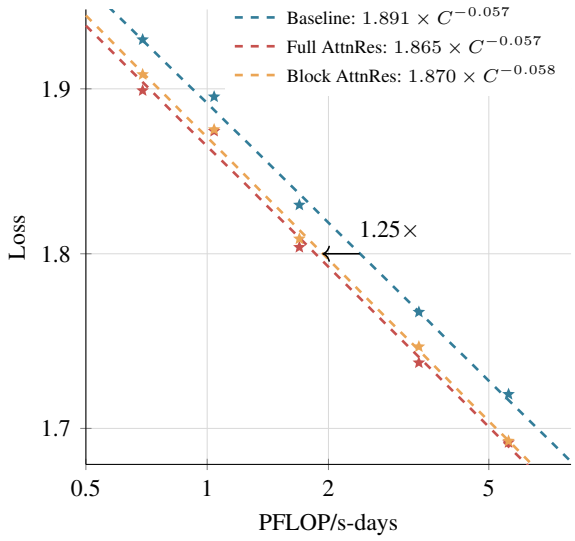


Figure 4: Scaling law curves for Attention Residuals. Both Full and Block AttnRes consistently outperform the baseline across all scales. Block AttnRes closely tracks Full AttnRes, recovering most of the gain at the largest scale.

PFLOP/s-days, Block AttnRes reaches 1.692 versus the Baseline's 1.714, equivalent to a $1.25\times$ compute advantage. The gap between Full and Block AttnRes narrows with scale, shrinking to just 0.001 at the largest size. We also list mHC(-lite) [64] in Table 2 for reference. Full AttnRes outperforms mHC, while Block AttnRes matches it at lower memory I/O per layer: $5.5d$ versus $34d$ for mHC with m=4 streams (Table 1).

5.2 Main Results

Training recipe. The largest models we study are based on the full Kimi Linear 48B configuration: 27 Transformer blocks (54 layers) with 8 out of 256 routed experts plus 1 shared expert, yielding 48B total and 3B activated parameters. This model applies Block AttnRes with 6 layers per block, producing 9 blocks plus the token embedding for a total of 10 depth-wise sources.

We follow the same data and training recipe as the Kimi Linear 1.4T-token runs [69]: all models are pre-trained with a 4096-token context window, the Muon optimizer [28], and a WSD (Warmup–Stable–Decay) learning rate schedule [16], with a global batch size of 8M tokens. Training of the final model proceeds in two stages: (i) a WSD pre-training phase on 1T tokens, followed by (ii) a mid-training phase on $\approx$400B high-quality tokens, following the annealing recipe of Moonlight [28].

After mid-training, we continue training with progressively longer sequence length of 32K tokens. Since our architecture uses hybrid KDA/MLA attention [69], where MLA operates without positional encodings (NoPE) [61], context extension requires no modifications such as YaRN [37] or attention temperature rescaling.

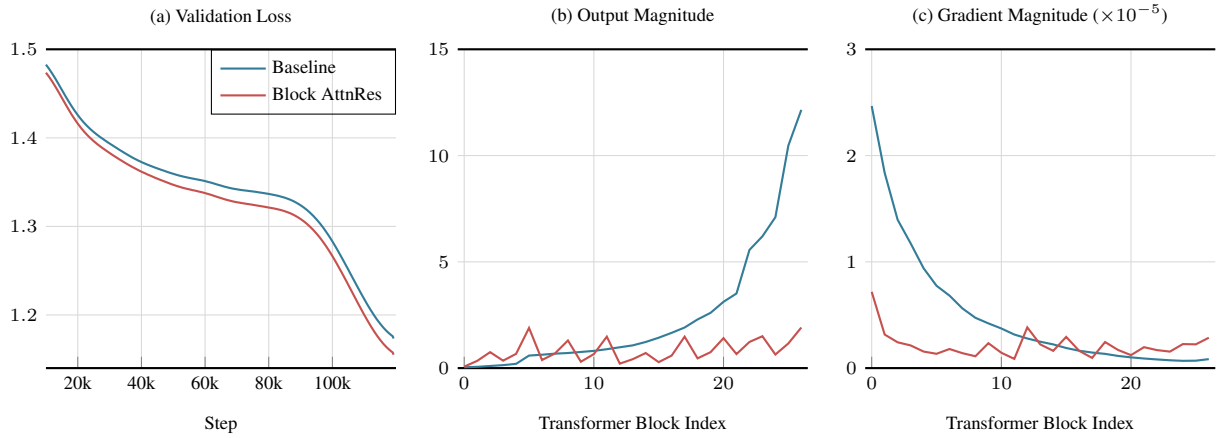


Figure 5: Training dynamics of Baseline and Block AttnRes. (a) Validation loss during training. (b) Each transformer block's output magnitude at the end of training. (c) Each transformer block's gradient magnitude.

Training dynamics. We compare the training dynamics of our final Baseline and Block AttnRes models over 1T tokens in Fig. 5.

- **Validation loss:** AttnRes achieves consistently lower validation loss throughout training, with the gap widening during the decay phase and resulting in a notably lower final loss.
- **Output magnitude:** The Baseline suffers from the PreNorm dilution problem [60, 27]: as hidden-state magnitudes grow monotonically with depth, deeper layers are compelled to learn increasingly large outputs from fixed-scale normalized inputs to remain influential. Block AttnRes confines this growth within each block, as selective aggregation at block boundaries resets the accumulation, yielding a bounded periodic pattern.
- **Gradient magnitude:** With all residual weights fixed to 1, the Baseline provides no means of regulating gradient flow across depth, leading to disproportionately large gradients in the earliest layers. The learnable $\mathrm{softmax}$ weights in Block AttnRes (Fig. 8) introduce competition among sources for probability mass, resulting in a substantially more uniform gradient distribution.

Table 3: Performance comparison of AttnRes with the baseline, both after the same pre-training recipe. Best per-row results are **bolded**.

		Baseline	AttnRes
General	MMLU	73.5	**74.6**
	MMLU-Pro	**52.2**	**52.2**
	GPQA-Diamond	36.9	**44.4**
	BBH	76.3	**78.0**
	ARC-Challenge	64.6	**65.7**
	HellaSwag	83.2	**83.4**
	TriviaQA	69.9	**71.8**
Math & Code	GSM8K	81.7	**82.4**
	MGSM	64.9	**66.1**
	Math	53.5	**57.1**
	CMath	84.7	**85.1**
	HumanEval	59.1	**62.2**
	MBPP	72.0	**73.9**
Chinese	CMMLU	82.0	**82.9**
	C-Eval	79.6	**82.5**

Downstream performance. Following the evaluation protocol of Kimi Linear [69], we assess both models across three areas (Table 3):

Table 4: Ablation on key components of AttnRes (16-layer model).

Variant		Loss
Baseline (PreNorm)		1.766
DenseFormer [36]		1.767
mHC [59]		1.747
AttnRes	Full	1.737
	w/ input-dependent query	1.731
	w/ input-independent mixing	1.749
	w/ sigmoid	1.741
	w/o RMSNorm	1.743
	SWA ($W = 1 + 8$)	1.764
	Block ($S = 4$)	1.746
	w/ multihead ($H = 16$)	1.752
	w/o RMSNorm	1.750

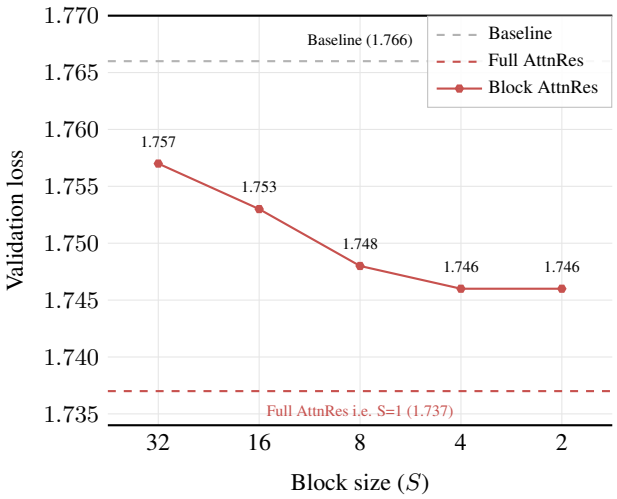


Figure 6: Effect of block size on validation loss (16-layer model).

- **Language understanding and reasoning**: MMLU [13], MMLU-Pro Hard [55], GPQA-Diamond [41], BBH [48], ARC-Challenge [6], HellaSwag [65], and TriviaQA [21].
- **Reasoning (Code and Math)**: GSM8K [7], MGSM [44], Math [25], CMath [14], HumanEval [5], and MBPP [1].
- **Chinese language understanding**: CMMLU [26] and C-Eval [19].

As shown in Table 3, Block AttnRes matches or outperforms the baseline on all benchmarks. The improvements are particularly pronounced on multi-step reasoning tasks such as GPQA-Diamond (+7.5) and Minerva Math (+3.6), as well as code generation such as HumanEval (+3.1), while knowledge-oriented benchmarks such as MMLU (+1.1) and TriviaQA (+1.9) also show solid gains. This pattern is consistent with the hypothesis that improved depth-wise information flow benefits compositional tasks, where later layers can selectively retrieve and build upon earlier representations.

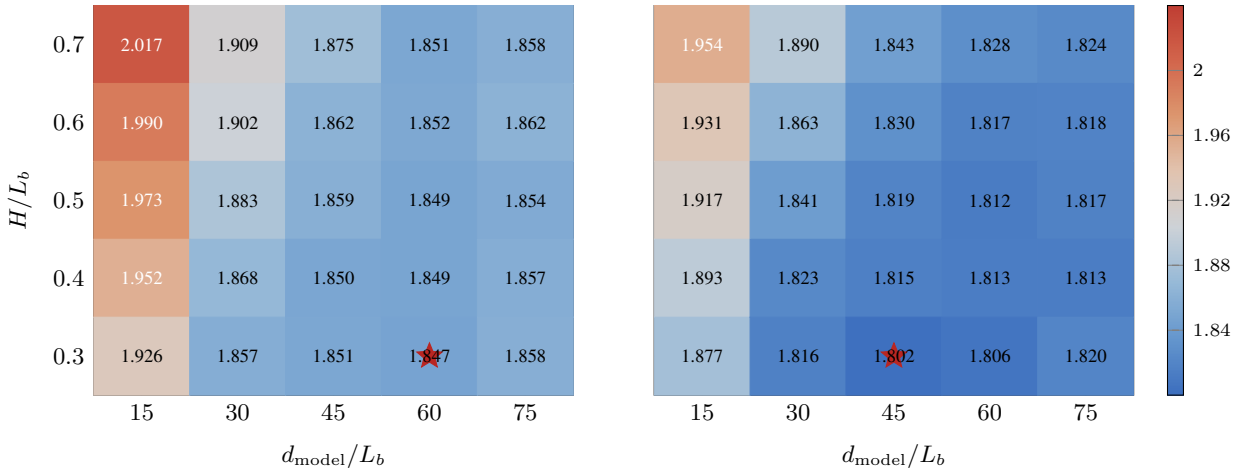
5.3 Ablation Study

We conduct ablation studies on the 16-head model from Table 2 to validate key design choices in AttnRes (Table 4). All models share identical hyperparameters and compute budget.

Comparison with prior methods. We compare AttnRes against the PreNorm baseline (loss 1.766) and two representative methods that generalize residual connections. DenseFormer [36] grants each layer access to all previous outputs but combines them with fixed, input-independent scalar coefficients; it shows no gain over the baseline (1.767), highlighting the importance of input-dependent weighting. mHC [59] introduces input dependence through m parallel streams with learned mixing matrices, improving to 1.747. AttnRes takes this further with explicit content-dependent selection via $\mathrm{softmax}$ attention: Full AttnRes achieves 1.737 and Block AttnRes 1.746, outperforming both methods with only a single query vector per layer.

Cross-layer access. We compare three granularities of cross-layer access. Full AttnRes follows directly from the time–depth duality (§ 3), applying attention over all previous layers, and achieves the lowest loss (1.737). A simple way to reduce its memory cost is sliding-window aggregation (SWA), which retains only the most recent $W{=}8$ layer outputs plus the token embedding; it improves over baseline (1.764) but falls well short of both Full and Block AttnRes, suggesting that selectively accessing distant layers matters more than attending to many nearby ones.

Block AttnRes offers a better trade-off: with block size $S{=}4$ it reaches 1.746 while keeping memory overhead constant per layer. Fig. 6 sweeps S across the full spectrum from $S{=}1$ (i.e. Full AttnRes) to increasingly coarse groupings. Loss degrades gracefully as S grows, with $S{=}2, 4, 8$ all landing near 1.746 while larger blocks ($S{=}16, 32$) move toward baseline. In practice, we fix the number of blocks to ≈ 8 for infrastructure efficiency (§ 4). As future hardware alleviates memory capacity constraints, adopting finer-grained block sizes or Full AttnRes represents a natural pathway to further improve performance.



(a) Baseline (b) Attention Residuals

Figure 7: Architecture sweep under fixed compute ($\approx 6.5 \times 10^{19}$ FLOPs, $\approx 2.3 \times 10^{8}$ active parameters). Each cell reports validation loss for a $(d_{\text{model}}/L_b,\ H/L_b)$ configuration, where $L_b = L/2$ is the number of Transformer blocks; the star marks the optimum.

Component design. We further ablate individual components of the attention mechanism:

- **Input-dependent query.** A natural extension is to make the query input-dependent by projecting it from the current hidden state. This further lowers loss to 1.731, but introduces a $d \times d$ projection per layer and requires sequential memory access during decoding, so we default to the learned query.
- **Input-independent mixing.** We removed the query and key and replaced them with learnable, input-independent scalars to weigh previous layers, which hurts performance (1.749 vs. 1.737).
- softmax **vs.** sigmoid**.** Replacing softmax with sigmoid degrades performance (1.741). We attribute this to softmax's competitive normalization, which forces sharper selection among sources.
- **Multihead attention.** We test per-head depth aggregation (H=16) on Block AttnRes, allowing different channel groups to attend to different source layers. This hurts performance (1.752 vs. 1.746), indicating that the optimal depth-wise mixture is largely uniform across channels: when a layer's output is relevant, it is relevant as a whole.
- RMSNorm **on keys.** Removing RMSNorm degrades both Full AttnRes (1.743) and Block AttnRes (1.750). For Full AttnRes, it prevents individual layers with naturally larger outputs from dominating the softmax. This becomes even more critical for Block AttnRes, as block-level representations accumulate over more layers and can develop large magnitude differences; RMSNorm prevents these from biasing the attention weights.

5.4 Analysis

5.4.1 Optimal Architecture

To understand how AttnRes reshapes optimal architectural scaling, we perform a controlled capacity reallocation study under a fixed compute and parameter budget. Our central question is whether AttnRes alters the preferred depth–width–attention trade-off, and in particular, given its potential strength on the depth dimension, whether it favors deeper models compared to conventional Transformer design heuristics. To isolate structural factors directly coupled to depth, we fix the per-expert MLP expansion ratio based on internal empirical observations ($d_{\text{ff}}/d_{\text{model}} \approx 0.45$). We further fix total training compute (FLOPs $\approx 6.5 \times 10^{19}$) and active parameters ($\approx 2.3 \times 10^{8}$), ensuring that any performance variation arises purely from architectural reallocation rather than overall capacity differences. Under this constrained budget, we enumerate 25 configurations on a 5×5 grid over $d_{\text{model}}/L_b \in \{15, 30, 45, 60, 75\}$ and $H/L_b \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$, where $L_b = L/2$ is the number of Transformer blocks and H the number of attention heads. The results are shown in Fig. 7.

Both heatmaps exhibit a shared pattern: loss decreases with growing d_{model}/L_b and shrinking H/L_b, and both methods reach their optima at $H/L_b \approx 0.3$. Despite this shared trend, AttnRes achieves a lower loss than the baseline in each of the 25 configurations, by 0.019–0.063. The most apparent difference lies in the location of the optimum: the baseline achieves its lowest loss at $d_{\text{model}}/L_b \approx 60$ (1.847), whereas AttnRes shifts it to $d_{\text{model}}/L_b \approx 45$ (1.802). Under a fixed

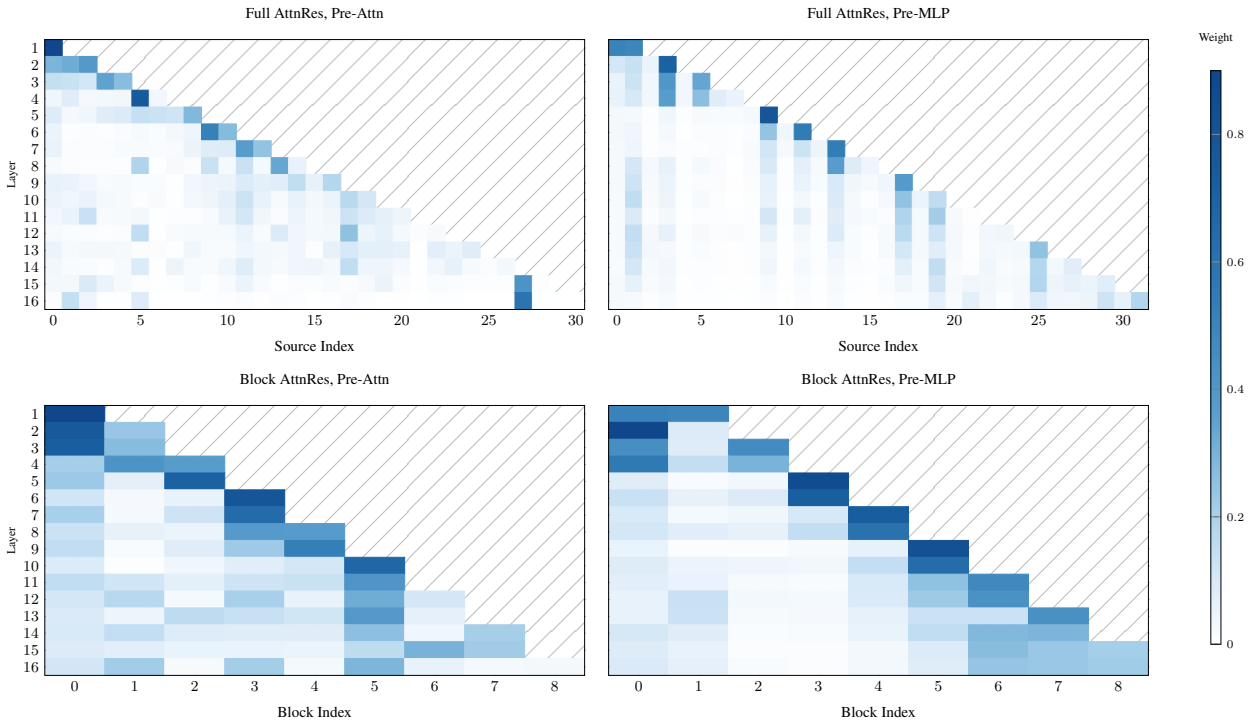


Figure 8: Depth-wise attention weight distributions for a 16-head model with full (top) and block (bottom) Attention Residuals, averaged over tokens. The model has 16 attention and 16 MLP layers. Each row shows how the lth attention (left) or MLP (right) layer distributes weight over previous sources. Diagonal dominance indicates locality remains the primary information pathway, while persistent weights on source 0 (embedding) and occasional off-diagonal concentrations reveal learned skip connections. Block attention ($N = 8$) recovers the essential structure with sharper, more decisive weight distributions.

parameter budget, a lower d_{model}/L_b corresponds to a deeper, narrower network, suggesting that AttnRes can exploit additional depth more effectively. We note that this preference for depth does not directly translate to a deployment recommendation, as deeper models generally incur higher inference latency due to their sequential computation [39]. Rather, this sweep serves as a diagnostic that reveals where AttnRes benefits most, and this depth preference can be factored into the architecture selection alongside inference cost.

5.4.2 Analyzing Learned AttnRes Patterns

We visualize the learned weights $\alpha_{i\to l}$ in Fig. 8 for the 16-head model (from Table 2) with both full and block (N=8) AttnRes. Each heatmap shows how the lth attention or MLP layer (rows) allocates its attention over previous sources (columns), with pre-attention and pre-MLP layers shown separately. We highlight three key observations:

- **Preserved locality.** Each layer attends most strongly to its immediate predecessor, yet selective off-diagonal concentrations emerge (e.g., layer 4 attending to early sources, layers 15–16 reaching back under the block setting), indicating learned skip connections beyond the standard residual path.

- **Layer specialization.** The embedding $\boldsymbol{h}_1$ retains non-trivial weight throughout, especially in pre-attention layers. Pre-MLP inputs show sharper diagonal reliance on recent representations, while pre-attention inputs maintain broader receptive fields, consistent with attention routing information across layers and MLPs operating locally.

- **Block AttnRes preserves structure.** Diagonal dominance, embedding persistence, and layer specialization all transfer from the full to the block variant, suggesting that block-wise compression acts as implicit regularization while preserving the essential information pathways.

Table 5: Comparison of residual update mechanisms. *Weight*: whether the mixing coefficients are architecture-fixed, learned-static (fixed after training), or input-dependent (dynamic). *Source*: which earlier representations layer l can access. Normalization is omitted from most formulas for clarity.

Method		Update rule	Weight	Source
Single-state recurrence: layer l receives only $\boldsymbol{h}_{l-1}$				
Residual [12]		$\boldsymbol{h}_l = \boldsymbol{h}_{l-1} + f_{l-1}(\boldsymbol{h}_{l-1})$	Fixed	$\boldsymbol{h}_{l-1}$
ReZero [2]		$\boldsymbol{h}_l = \boldsymbol{h}_{l-1} + \alpha_l \cdot f_{l-1}(\boldsymbol{h}_{l-1})$	Static	$\boldsymbol{h}_{l-1}$
LayerScale [50]		$\boldsymbol{h}_l = \boldsymbol{h}_{l-1} + \mathrm{diag}(\boldsymbol{\lambda}_l) \cdot f_{l-1}(\boldsymbol{h}_{l-1})$	Static	$\boldsymbol{h}_{l-1}$
Highway [45]		$\boldsymbol{h}_l = (1-\boldsymbol{g}_l) \odot \boldsymbol{h}_{l-1} + \boldsymbol{g}_l \odot f_{l-1}(\boldsymbol{h}_{l-1})$	Dynamic	$\boldsymbol{h}_{l-1}$
DeepNorm [54]		$\boldsymbol{h}_l = \mathrm{Norm}(\alpha\, \boldsymbol{h}_{l-1} + f_{l-1}(\boldsymbol{h}_{l-1}))$	Fixed	$\boldsymbol{h}_{l-1}$
KEEL [4]		$\boldsymbol{h}_l = \mathrm{Norm}(\alpha\, \boldsymbol{h}_{l-1} + f_{l-1}(\mathrm{Norm}(\boldsymbol{h}_{l-1})))$	Fixed	$\boldsymbol{h}_{l-1}$
Multi-state recurrence: layer l receives m streams				
SiameseNorm [27]		$\boldsymbol{h}_l^1 = \mathrm{Norm}(\boldsymbol{h}_{l-1}^1 + \boldsymbol{y}_{l-1});\ \boldsymbol{h}_l^2 = \boldsymbol{h}_{l-1}^2 + \boldsymbol{y}_{l-1}$	Fixed	2 streams
HC/mHC [72, 59]		$\mathbf{H}_l = \mathbf{H}_{l-1}\mathbf{A}_l + f_{l-1}(\mathbf{H}_{l-1}\boldsymbol{\alpha}_{l-1})\, \boldsymbol{\beta}_{l-1}^\top$	Dynamic	m streams
DDL [67]		$\mathbf{H}_l = (\mathbf{I} - \beta_l \boldsymbol{k}_l \boldsymbol{k}_l^\top)\mathbf{H}_{l-1} + \beta_l \boldsymbol{k}_l \boldsymbol{v}_l^\top$	Dynamic	d_v streams
Cross-layer access: layer l can access individual earlier-layer outputs				
DenseNet [17]		$\boldsymbol{h}_l = \mathrm{ConvPool}([\boldsymbol{h}_1;\ f_1(\boldsymbol{h}_1);\ \ldots;\ f_{l-1}(\boldsymbol{h}_{l-1})])$	Static	$[\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{l-1}]$
DenseFormer [36]		$\boldsymbol{h}_l = \alpha_{0\to l}\, \boldsymbol{h}_1 + \sum_{i=1}^{l-1} \alpha_{i\to l}\, f_i(\boldsymbol{h}_i)$	Static	$[\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{l-1}]$
MRLA [10][1]		$\boldsymbol{h}_l = \sum_{i=1}^{l-1} \sigma\big(\mathrm{ConvPool}(f_{l-1}(\boldsymbol{h}_{l-1}))\big)^\top \sigma\big(\mathrm{ConvPool}(f_i(\boldsymbol{h}_i))\big)\, \mathrm{Conv}(f_i(\boldsymbol{h}_i))$	Dynamic	$[\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{l-1}]$
AttnRes (ours)	Full[2]	$\boldsymbol{h}_l \propto \sum_{i=0}^{l-1} \phi(\boldsymbol{w}_l, \boldsymbol{k}_i)\, \boldsymbol{v}_i$	Dynamic	$[\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{l-1}]$
	Block[3]	$\boldsymbol{h}_l \propto \sum_{i=0}^{n-1} \phi(\boldsymbol{w}_l, \boldsymbol{k}_i)\, \boldsymbol{v}_i + \phi(\boldsymbol{w}_l, \boldsymbol{k}_n^j)\, \boldsymbol{v}_n^j$	Dynamic	$[\boldsymbol{b}_0, \ldots, \boldsymbol{b}_{n-1}, \boldsymbol{b}_n^j]$

[1] ConvPool: pooling operation followed by convolution (channel projection).
[2] $\phi(\boldsymbol{q}, \boldsymbol{k}) = \exp\big(\boldsymbol{q}^\top \mathrm{RMSNorm}(\boldsymbol{k})\big);\ \boldsymbol{k}_i = \boldsymbol{v}_i;\ \boldsymbol{v}_0 = \boldsymbol{h}_1,\ \boldsymbol{v}_{i\geq 1} = f_i(\boldsymbol{h}_i)$. softmax jointly normalized over all sources.
[3] Same ϕ and normalization as Full; $\boldsymbol{v}_i = \boldsymbol{b}_i,\ \boldsymbol{v}_n^j = \boldsymbol{b}_n^j$.

6 Discussions

6.1 Sequence-Depth Duality

Residual connections propagate information over depth via a fixed recurrence $\boldsymbol{h}_l = \boldsymbol{h}_{l-1} + f_{l-1}(\boldsymbol{h}_{l-1})$, much as RNNs propagate information over time. Test-Time Training (TTT) [46] formalizes the sequence side of this analogy (cf. Fast Weight Programmers [43, 32]), casting each recurrent step as gradient descent on a self-supervised loss:

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta\, \nabla \ell(\mathbf{W}_{t-1};\, \boldsymbol{x}_t), \tag{9}$$

where a slow network parameterizes ℓ and the state $\mathbf{W}$ is updated once per token. When f is linear, this reduces to vanilla linear attention $\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{k}_t \boldsymbol{v}_t^\top$. The standard residual exhibits the same additive form along depth, with $\boldsymbol{h}_l$ serving as the state and each layer f_l acting as one "gradient step."

As noted by [4], this duality extends to richer variants (Table 5). Data-dependent gates on the sequence side [47, 63] correspond to Highway networks [45] on the depth side; the delta rule [42, 62, 69] corresponds to DDL [67]; and MRLA [10] mirrors GLA's [63] gated linear attention. These methods all refine the recurrent update while remaining within the recurrence paradigm. AttnRes goes a step further and replaces depth-wise recurrence with direct cross-layer attention, just as Transformers replaced temporal recurrence with self-attention. Since the number of layers in current architectures remains well within the practical regime of softmax attention, we adopt vanilla depth-wise attention. Incorporating more expressive yet memory-efficient (e.g. linear-complexity) alternatives is a natural direction for future work.

6.2 Residual Connections as Structured Matrices

The residual variants discussed above can all be viewed as weighted aggregations over previous layer outputs. We formalize this with a *depth mixing matrix* $\mathbf{M} \in \mathbb{R}^{L\times L}$, where $\mathbf{M}_{i\to l}$ is the weight that layer l assigns to the output of layer i. The variants differ in how these weights arise (fixed, learned, or input-dependent) and whether $\mathbf{M}$ is constrained to low rank or allowed to be dense. The semiseparable rank of $\mathbf{M}$ [8] offers a unified lens for comparing them.

Concretely, the input to layer l is $\boldsymbol{h}_l = \sum_{i=0}^{l-1} \mathbf{M}_{i\to l}\, \boldsymbol{v}_i$, where $\boldsymbol{v}_0 = \boldsymbol{h}_1$ (embedding) and $\boldsymbol{v}_i = f_i(\boldsymbol{h}_i)$ for $i \geq 1$. Fig. 9 visualizes $\mathbf{M}$ for representative methods; we derive each below.